Useful-Work Coin: Design v3

Nov 5, 2026 · @Sid

Summary

The design is Pearl's cuPOW lottery, unchanged, plus a verified job market in which SEDULity-style sampled checks release a withheld fee. A share of issuance buys public-good inference. Nothing is ever paid per job.

Three results drive it. Each is derived in the doc and checked with a simulator (sim.py, sent with this doc):

1. **No per-job subsidy survives self-dealing.** A miner can post jobs to itself, so any protocol payment tied to a job is only safe if it is no larger than what that job burns. Usefulness has to come from cuPOW's dual revenue (the same int8 GEMMs serve a client and mine), from real client fees, and from jobs the protocol itself chooses.
2. **A public-good pool funded by a share ρ of issuance moves a share ρ / (1 − λ) of GPUs onto useful work.** λ is how many tickets a GPU still mines while running inference, relative to pure noise mining. The cost is a hashrate drop of ρ, capped at 1 − λ.
3. **Verification can cost well under 1% of the job.** If the fee is withheld until checks pass and carries a margin over cost, two random stage samples make skipping work a losing bet. The earlier drafts assumed about 15%.

What changed from the earlier drafts

Earlier drafts	This version	Why
Bind tickets to jobs; a premium M for job tiles	No binding, no premium	A premium can only refund the job's own burn (result 1), so it adds rules and buys nothing
Burn part of every fee	No burn; a small treasury fee funds forced-error jackpots	Burn existed only to cap the premium
Sample 2–10% of stages, 3 verifiers	2 samples per verifier, 2–3 verifiers, client may buy more	Withheld fee works as collateral (result 3)
Lazy verifiers caught by planted faults	Commit-reveal stops copying; Truebit-style forced errors keep outside challengers watching	Planted faults could not catch collusion with a real provider

Earlier drafts	This version	Why
Any inference job	Prefer jobs that are mostly large GEMMs (high λ)	Low-λ serving cannot match noise mining, and Basu found no identifiable inference code in Pearl's dominant miner

This is my design, not a reviewed protocol. The simulator tests the incentive arithmetic under stated assumptions. It does not test networking, cryptography or real GPU behaviour.

How cuPOW and SEDULity combine

cuPOW decides who makes blocks. SEDULity's verification ideas decide who gets paid for jobs: the stage as the unit of work, sampled rechecks, payment held until checks pass, and a committee drawn from hashed deposits. SEDULity's lottery, which makes each training stage a ticket, is not used for blocks.

Why cuPOW tickets, not stage tickets

A SEDULity ticket is a hash of a stage's resulting weights and the previous block hash. A node can check that hash immediately, but not whether the weights came from honest training, which only a later sampled check settles. A miner who doesn't care about rewards, such as an attacker, could hash weights it never trained, unless every winning stage is verified before its block is accepted.

A cuPOW ticket has no such gap. Every node checks the noisy MatMul jackpot when the block arrives. So consensus stays with cuPOW, and the delayed, sampled checks move to where delay is fine: payment.

The payout rule

The protocol never pays anything per job, except for jobs it chose itself. The argument:

1. Let π(J) be any extra payment, in coins or better ticket odds, for completing a client job J.
2. The protocol cannot see who the client is, so a miner can post J to itself. Its net is π(J) minus whatever J burns or wastes.
3. Self-dealing is therefore profitable whenever π(J) > burn(J), and it grows until the subsidy is drained into jobs nobody needed.
4. Capping π(J) at burn(J) makes an honest provider's premium exactly the client's burn handed back: a transfer, not a subsidy.

Simulator part B shows it. With a 4× ticket premium, a self-dealer nets zero when the fee sits at the burn floor, and profits by the gap when the fee is half the floor.

Three sources of usefulness survive the rule:

- **Dual revenue (cuPOW).** A provider's int8 GEMMs for a client also mine. The client only has to cover the tickets lost to non-GEMM work, a share $1 - \lambda$ of the GPU's mining revenue for that time.
- **Client fees.** Real demand needs no protection, because nothing is added to it.
- **Pool jobs.** The protocol chooses them, so a miner cannot post them to itself.

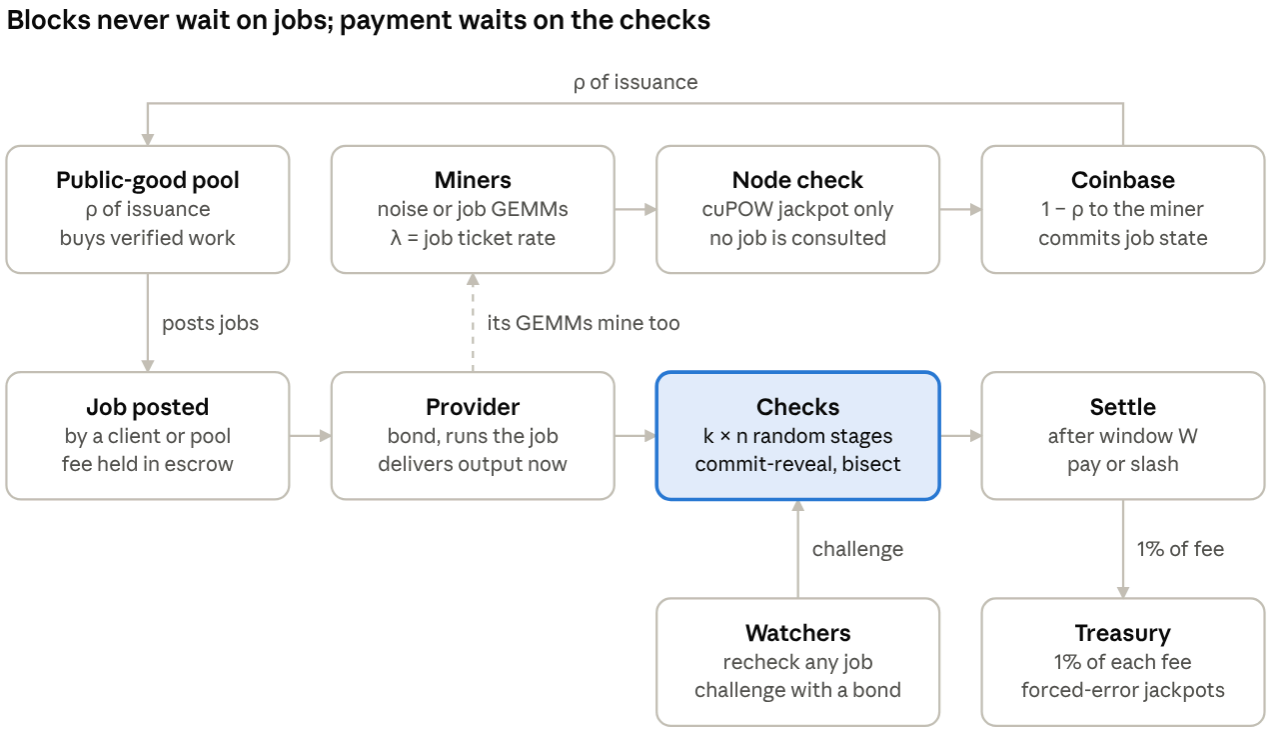
Where each paper lands

Source	Role in this version
cuPOW / Pearl	Consensus and dual revenue, unchanged
SEDULity	Stage as verification unit; payment held as collateral; committee drawn from hashed deposits
Proof of Training	Challenge window, slashing, challenger bounty, commit-reveal; its L1/L2 split deferred
Proof of Useful Intelligence	Roles; public-good jobs; price that rises when providers are scarce
HadAgent	Signed proof records; trusted and untrusted tiers; benchmark audits as canary jobs
Barbaccia et al.	Three-layer split; stake condition (gain ÷ detection × penalty); fees paid in the coin; post-quantum signatures
PoUW for AI (Lihu et al.)	Clients pay workers; precedent for rechecking the lucky step
PoGO	Version 2 training jobs: Merkle commitments over quantized steps
Coin.AI, PoLe, WekaCoin, PoFLSC, PoDL survey	Version 2 competition jobs scored on held-out data; never consensus
BlockML notes	Not used: a trusted task supplier sets difficulty
Truebit	Forced errors with jackpots, so outside challengers keep checking

Source	Role in this version
Verde	Disputes narrowed to one operation; reproducible operators across hardware
I-LLM	Evidence that integer-only LLM inference is feasible (W4A4 with negligible loss, per its abstract)
Basu	The usefulness gap; the value destruction ratio as a metric

Architecture

The coin is one chain built from Pearl's code, with a job module inside the full node. The three layers follow Barbaccia et al.: consensus decides blocks, the job layer decides payment, and the economics layer funds demand and watchers.



v3 architecture · consensus, jobs and economics

The only links between consensus and jobs are the dashed one, where a provider's GEMMs double as mining, and the coinbase's split to the pool. Neither makes a block's validity depend on a job.

Each role, and what it puts at stake:

- **Clients and the pool** post jobs and escrow the fee.

- **Providers** post a bond, run jobs and mine on the same GPUs.
- **Verifiers** post a deposit and recheck random stages.
- **Watchers** can be anyone, post a bond per challenge, and earn forced-error jackpots.
- **Full nodes** check blocks and settle disputes down to one operation.

There is no separate L2 in version 1. Proof of Training's L1/L2 split can wait until job traffic outgrows the chain.

Consensus layer

The chain is a new network built from Pearl's code, with cuPOW mining left exactly as it is. Only two consensus rules are added, both on the coinbase:

1. **Issuance split.** The coinbase must pay a share ρ of the block subsidy to the public-good pool account; the rest goes to the miner.
2. **Job-state commitment.** The coinbase must commit the root of the job module's state tree, as SegWit commits witness data.

Everything Pearl already has stays: btcd node, NoisyGEMM mining, the zk proof-of-work circuit (Plonky2/STARKy), XMSS signatures and the vLLM miner. Basu reports that Pearl's verifier checks matrix entries lie in {−64, …, 64} but not that they come from a model. This design keeps it that way: block validity never depends on usefulness.

What miners run

A GPU either noise-mines (random matrices, the most tickets per second) or runs a paid job whose int8 GEMMs also mine. **λ** is the ratio of tickets per second while running the job to tickets per second while noise-mining. Time spent in attention softmax, norms, memory-bound decoding or idle waits produces no tickets and lowers λ.

λ sets everything downstream. The fee a client must pay is the share 1 − λ of mining income the provider gives up. The public pool's reach is ρ / (1 − λ). So version 1 accepts only job types that are mostly large matrix multiplies:

Job type	Expected λ	How it is checked	Version
Embeddings, reranking, classification over large batches	High: batched forward passes are compute-bound GEMMs	Recompute sampled (sequence, layer group) stages	1
Log-likelihood scoring and eval suites	High, for the same reason	Same	1

Job type	Expected λ	How it is checked	Version
Large-batch offline generation	Medium, rising with batch size	Teacher-forced recompute of sampled sequences	1
Interactive chat	Low: single-stream decoding is memory-bound	Not settled on-chain; provider and client deal directly	—
Fine-tuning and training	Medium to high	PoGO-style step commitments	2

The λ values are expectations from how these workloads use a GPU, not measurements. Measuring them is the first testnet task.

Two constraints from mining on real models

- **Value range.** For a model's GEMMs to count as mining, its weights and activations must be quantized into {−64, …, 64}, about 7 bits plus sign. I-LLM reports integer-only inference at W4A4 with negligible accuracy loss, so a wider range is plausible. It still has to be measured per model.
- **Integer-only runtime.** Integer addition is exact and order-independent, so an integer-only model gives bit-identical results however the work is batched, tiled or split between prefill and decode. That is what lets a verifier check generated text by running prompt plus output as one prefill (teacher forcing). The provider samples tokens with a random generator seeded by H(job, sequence, position), so a verifier reproduces each choice exactly.

Job layer

A job is a batch inference task whose fee sits in escrow until a small random sample of its stages is recomputed and a challenge window passes. A stage is one batch of sequences through one group of layers, so a job has S = batches × layer groups stages.

Lifecycle

1. **Post.** A client, or the pool, sends JOB_POST: model, input root, layer groups, fee into escrow, deadline, and the verification level (k verifiers, n samples each; minimum 2 and 2).
2. **Accept.** A provider sends JOB_ACCEPT with a bond of at least half the fee. Trusted-tier providers may post less.

3. **Run and deliver.** The provider runs the job, mining on its GEMMs, and sends the outputs to the client off-chain at once. The client checks them against the output root posted next.
4. **Commit.** TRACE_COMMIT carries a salted commitment to every stage's output root, plus the final output root.
5. **Draw.** The hash of the next block picks k verifiers from bonded stake, by sorting H(block hash, address), excluding the provider (SEDULity's committee draw).
6. **Sample privately.** Each verifier derives its n stage positions from a secret salt and commits H(salt) on-chain. The provider learns the positions only when asked for data, after its roots are fixed, so it cannot steer the sample.
7. **Serve data.** The provider must supply each sampled stage's input within D blocks. It can recompute any input from the job's inputs, because the runtime is deterministic, so it doesn't need to store activations. Missing data means a slash.
8. **Recompute and commit.** Each verifier recomputes its stages with the reference runtime and commits H(output roots, own address, salt) before the provider reveals.
9. **Reveal.** The provider reveals its roots, then the verifiers reveal theirs and their salts. Anyone can check that the positions match the salts.
10. **Dispute.** On a mismatch, or on anyone's bonded CHALLENGE during the window W, the two sides bisect inside the stage to one int8 operation. Full nodes recompute it on CPU (Verde's refereed-delegation pattern), and the loser is slashed.
11. **Settle.** After W, the provider gets the fee minus the verifier and treasury shares, and its bond back.

Why a tiny sample is enough

If a provider skips m of S stages, it saves m × c, where c is what one stage costs it (for a provider that mines, the extra income noise mining would earn in that time). If caught, it loses the withheld fee F and its bond B. Compared with honest work, its expected gain is:

```
G(m) = m\,c - P(m)\,(F + B)
```

P(m), the chance that a random sample hits a skipped stage, is concave in m and equals m / S with a single sample. So G is convex and its maximum is at m = 1 or m = S. Both ends are non-positive whenever:

```
F + B \ge S\,c
```

In words: **if the withheld fee plus the bond covers the provider's full cost of the job, skipping any part of it loses money, even with one sample.** Any margin makes the loss strict. This is SEDULity's withheld reward and Barbaccia's stake condition working as one.

Simulator part C confirms it: break-even at fee = cost with one sample and no bond, and strictly negative with a bond or a 20% margin.

More samples (2 × 2 by default) cover what the argument leaves out: providers who don't care about money, gains other than saved compute such as tampering with chosen outputs, and honest bugs. Clients with such risks buy higher k and n. At S = 1,000 stages, 2 × 2 samples cost 0.4% of the job's compute.

Keeping verifiers and challengers honest

- **Copying:** stopped by commit-reveal; a verifier commits before seeing the provider's roots.
- **Collusion with the provider:** a fully faked job passes only if all k drawn verifiers collude and no outside challenger rechecks it. With colluding stake q, that happens with probability $q^k \times (1 - w)$, where w is the chance a watcher checks (simulator part D).
- **Idle challengers (Truebit's forced errors):** the treasury posts and serves a few decoy jobs from fresh addresses, so they look like client jobs, and puts a wrong root into some of them, committed in advance under a secret revealed after W. No client is harmed, because the treasury is the decoy's client. Whoever catches one wins a jackpot. Watchers cannot tell decoys from client jobs, so checking any job pays.
- **Canary jobs (HadAgent):** pool jobs with known answers, checked in full. They catch model substitution at certainty.
- **Tiers (HadAgent):** after 5 clean jobs a provider's minimum bond falls; after 2 failures it rises again. Tiers change the bond, never the sample count.

Transactions

Type	Key fields
MODEL_REG	weights_root, reference runtime hash, layer groups, registrant bond
JOB_POST	model_id, input_root, batches, fee, deadline, k, n
JOB_ACCEPT	job_id, provider, bond
TRACE_COMMIT	job_id, salted per-stage output commitments, output_root
VERIFY_COMMIT, VERIFY_REVEAL	job_id, H(salt), H(outputs, address, salt); then the outputs and salt
TRACE_REVEAL	job_id, roots and salts for the requested stages

Type	Key fields
CHALLENGE, BISECT	job_id, stage, bond; one bisection step per transaction
SETTLE	job_id

Inputs are visible to the verifiers and challengers who check them, so version 1 suits public data and clients who accept that. Confidential jobs would need secure enclaves or similar hardware, which this design does not cover.

Economics

The one dial between security and usefulness is ρ, the share of issuance sent to the public-good pool. Every point of ρ moves GPUs onto useful work at a rate set by λ, and costs about one point of hashrate.

What a client has to pay

A provider serves a job instead of noise-mining when the fee covers what it gives up. Per GPU-second:

```
\text{fee} \;\ge\; (1 - \lambda)\cdot\text{mining income} \;+\;
\text{verifier and treasury shares}
```

Electricity and hardware are paid for either way, since both options run the GPU. At λ near 1, inference costs a client little more than verification. At low λ it costs about what noise mining would earn, consistent with Basu finding no identifiable inference code in Pearl's dominant miner.

The public-good pool

The pool spends its share ρ of each block's subsidy on verified work units from pool jobs, splitting the budget across the units delivered. Fewer providers means more per unit: PoUI's dynamic reward, with no price to set. Providers move from noise mining to pool jobs until both pay the same.

With all issuance R per block, N identical GPUs, a share υ of them on pool jobs and hashrate H measured in noise-GPU units, a GPU's income per block is:

```
\underbrace{(1-\rho)\frac{R}{H}}_{\text{noise}} \;=\; \underbrace{(1-
\rho)\frac{\lambda R}{H} + \frac{\rho R}{uN}}_{\text{pool job}}, \qquad H =
N\bigl(1 - u(1-\lambda)\bigr)
```

Solving gives:

$$u = \min\!\Bigl(1,\ \frac{\rho}{1-\lambda}\Bigr), \qquad \frac{H}{N} = 1 - \min(\rho,\ 1-\lambda)$$

Three consequences:

- **A pool is cheap where λ is high.** At λ = 0.9, ρ = 10% puts every GPU on pool work while hashrate falls 10%.
- **The security cost does not depend on λ until saturation.** Diverting ρ of issuance costs ρ of hashrate, because the pool money ends up paying for exactly the tickets that pool work gives up.
- **Raising λ is the cheapest lever.** Better int8 kernels and GEMM-heavy job types buy useful work at no cost to security.

This assumes a fixed GPU count, identical GPUs and risk-neutral providers, and it ignores client demand, which only adds useful work. Simulator part A checks the formula, and part A2 relaxes the identical-GPU assumption.

Paying verifiers and watchers

- **Verifiers** spend k × n / S of the job's compute, 0.4% at S = 1,000 with 2 × 2 samples. They receive 1% of the fee, with a floor computed from the declared stage cost, so small jobs (low S) still pay enough. Verifiers also mine on the GEMMs they recompute, so their cost and the fee both scale with 1 − λ and the ratio holds.
- **Watchers** are paid only through forced-error jackpots. If they check a share w of jobs with sampled checks, total jackpots need to cover about w × (cost of one sampled check). With w = 10%, 2 samples and S = 1,000, that is about 0.02% of job compute, well inside a 1% treasury share. Decoy jobs add their own compute: about 0.1% of job volume at one decoy per thousand jobs.

Starting values

Parameter	Starting value	Basis
Pool share ρ	10% of issuance	Moves up to 10% / (1 − λ) of GPUs, at 10% hashrate cost; tune once λ is measured
Fee split	Provider 98%, verifiers 1% (with a floor), treasury 1%	Overheads above
Verification	k = 2 verifiers, n = 2 samples each; clients may buy more	Condition F + B ⩾ S × c; extra samples for robustness

Parameter	Starting value	Basis
Provider bond	At least 50% of the fee; lower for the trusted tier	Puts F + B at 1.5× the fee, a margin over cost
Challenge window W	About one day of blocks	Time for any watcher to recompute a stage
Data deadline D	A few blocks	Recompute-on-demand is fast
Forced errors	Decoy jobs that look like client jobs; Truebit cites about one in a thousand tasks	Sized to the watch rate wanted
Fees	Paid in the coin only	Barbaccia's closed-loop idea; creates demand for the token

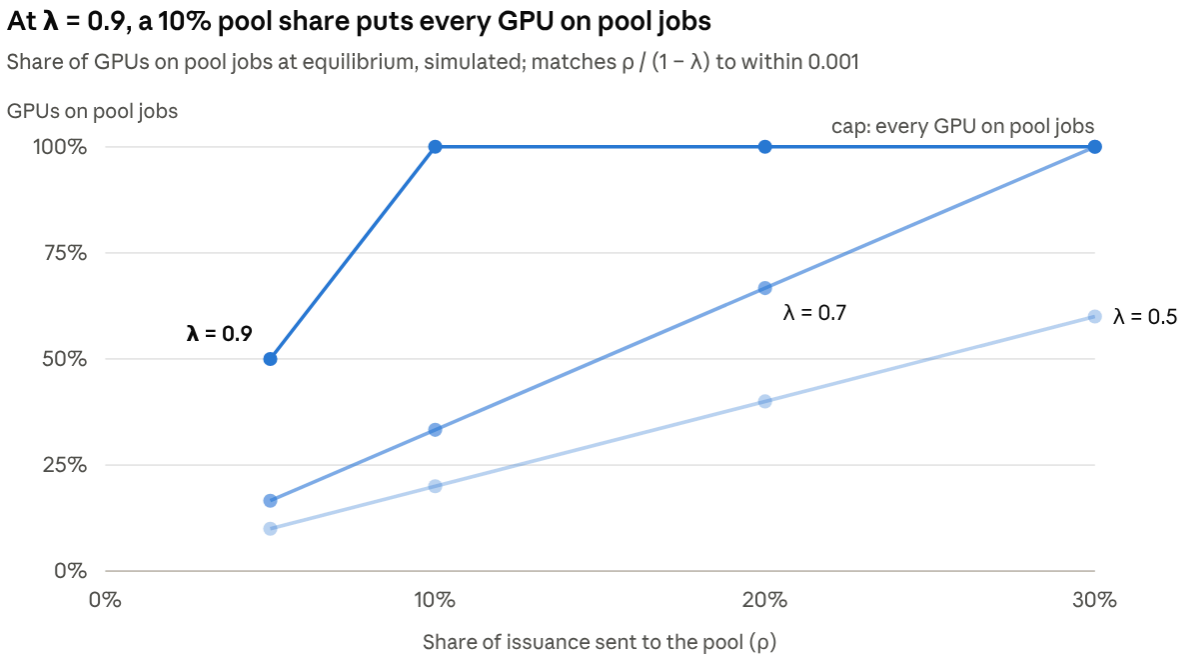
No burn is needed. With no per-job subsidy there is nothing for self-dealing to capture.

Simulator results

Under its assumptions, the simulator confirms all four claims. The pool formula matches an agent-based run to within 0.001, a skipped stage never pays once fee plus bond covers cost, and collusion odds match q^k × (1 − w). The script is sim.py (Python, standard library only, about 5 seconds to run), sent alongside this doc.

A. Pool equilibrium

Each GPU repeatedly chooses noise mining or pool jobs, whichever pays more given what the others do.



sim.py part A · 1,000 identical GPUs, 2,000 rounds of best response

Hashrate fell by exactly ρ in every unsaturated case, and by 1 − λ once every GPU had moved; at λ = 0.9 it stayed at 90% for ρ = 20% and 30%.

A2. Mixed GPUs. When each GPU's λ varies by ±0.2 around the mean, the pool attracts the GPUs that are best at inference first:

ρ	Mean λ	GPUs on pool jobs	Identical-GPU formula	Hashrate kept
10%	0.5	25.6%	20.0%	91.1%
20%	0.5	44.1%	40.0%	83.1%
10%	0.7	40.3%	33.3%	92.9%
20%	0.7	62.9%	66.7%	86.2%

With mixed hardware, a small pool reaches more GPUs and costs less hashrate than the formula says. Near saturation it reaches slightly fewer, because the worst inference GPUs stay on noise.

B. Self-dealing a premium

With a 4× ticket premium and a 10% burn, a miner buying jobs from itself netted +0.00017 per job at the burn floor. That is within sampling noise of the expected zero (standard error about 0.0003). At half the floor it netted +0.0062, matching the expected +0.0061. This is why the design has no premium.

C. Skipping stages

S = 1,000 stages; the worst case over every m from 1 to 1,000:

Fee ÷ cost	Bond ÷ cost	Samples	Best cheat's gain	Check overhead
1.0	0	1	0 (break-even)	0.1%
1.0	0.5	1	−0.5 stage-costs	0.1%
1.0	0.5	4	−5.0	0.4%
1.2	0	2	−1.4	0.2%
1.2	0.5	4	−5.8	0.4%

A Monte Carlo spot check of the detection formula gave 0.1430 against an exact 0.1428 (m = 50, 3 samples).

D. Collusion

A fully faked job passes with probability q^k × (1 − w); simulation agreed to within 0.001 in all 18 cases tried:

Colluding stake q	k = 2	k = 3	k = 5
10%	1.0%	0.1%	0.001%
33%	10.9%	3.6%	0.39%
50%	25.0%	12.5%	3.1%

These are without watchers (w = 0); watchers checking 10% of jobs cut each by a tenth. At k = 2, a colluding third of stake passes about one fake job in nine, so the k = 2 default only suits low-value jobs. Clients with valuable jobs should buy k = 5.

What the simulator does not test: networking, latency, cryptography, real GPU throughput, actual λ values, risk-averse or irrational actors, block withholding, or a GPU count that changes with revenue.

Security analysis

The chain is as secure as Pearl at the same hashrate, and the pool lowers that hashrate by ρ. The job layer can lose clients' money but can never make an invalid block valid.

Attack	Mitigation	What remains
51% attack on the chain	cuPOW unchanged; block validity never depends on jobs	An attacker needs (1 − ρ) of the GPUs it would need with no pool
Self-dealing	No per-job payment exists to capture	Nothing
Skipping stages to save compute	Fee withheld; bond; $F + B \geq S \times c$	Providers driven by something other than money
Tampering with chosen outputs	Client buys higher k, n and bond; client checks outputs against output_root; open challenges	Gain is not bounded by compute, so a small sample gives weak odds on one item
Running a cheaper model	Every stage output then differs, so any sample catches it; canary jobs	Nothing, given bit-exact checks
Verifier collusion	Committee drawn from stake after TRACE_COMMIT; watchers	Fake passes with probability $q^k \times (1 - w)$; weak at k = 2 against a large coalition
Redrawing the committee by withholding blocks	A provider who mines can drop its own block to re-roll, losing that block's reward each time	Draw from the hash $d \geq 2$ blocks later for jobs worth more than a block reward
Steering which stages are sampled	Verifiers pick positions from private salts after roots are committed	Nothing
Withholding data	Serve within D blocks or be slashed	Settlement can be delayed by up to D
Griefing with false challenges	Challenger bond, lost on a failed dispute	Cost of the bond only
False slashing from nondeterminism	Integer-only runtime; disputes end in a CPU-recomputed int8 operation	A bug in the reference runtime; needs cross-GPU test vectors

Attack	Mitigation	What remains
Insider front-running of forced errors	Forced-error secret held by a threshold of keyholders	An insider could still claim jackpots and starve honest watchers; known Truebit weakness
Capture of the public pool	Fixed job categories, public job lists, per-category caps, open race for units	Who picks categories remains a trust point
Input privacy	None in version 1	Sampled inputs are seen by verifiers

The limit that matters most

Usefulness is economic, not cryptographic. The share of GPUs doing useful work equals client demand plus $\rho / (1 - \lambda)$, and nothing in the protocol can raise it further without paying self-dealers.

The design can measure it, though. Settled job compute divided by the compute implied by the network's difficulty can be published every block, in the spirit of Basu's value destruction ratio.

Two numbers decide whether the coin is worth launching, and no paper or simulation supplies either:

- **λ for real models** quantized to $\{-64, \ldots, 64\}$.
- **The quality loss** from that quantization.

Build plan

Measure λ and quantization quality before writing any chain code; together they decide whether the coin can do useful work at all.

1. **Go or no-go: λ and quality.** Quantize one open embedding model and one open LLM to $\{-64, \ldots, 64\}$ with integer-only nonlinearities. Measure tickets per second on Pearl's miner for noise and for each job type in the table, and measure eval loss against the original model. Stop if λ is low on every job type, or the quality loss is unacceptable.
2. **Reference runtime.** Build integer-only kernels for GPU and CPU. Exit when outputs match bit for bit across two GPU families and CPU, for prefill, decode and teacher-forced runs.

3. **Job module.** Add the state tree, coinbase commitment, issuance split and the transaction types to Pearl's btcd fork. Exit when property tests of escrow, bonds and settlement pass.
4. **Verification.** Build the committee draw, private-salt sampling, commit-reveal, serve-or-slash and bisection to one operation. Exit when every fault injected in a test suite is caught and the right party is slashed.
5. **Pool and watchers.** Add pool budget accounting, unit pricing, forced errors with threshold keys, and canary jobs.
6. **Simulator, extended.** Add a GPU count that responds to revenue, risk-averse providers, block withholding and measured λ values to sim.py. Retune ρ, k, n and bonds.
7. **Public testnet.** Run real batch jobs from outside clients through the full flow.

Repository layout

```
chain/      Pearl btcd fork + jobstate/ (Go)
miner/      Pearl vLLM miner + integer-only job path (Python, CUDA)
runtime/    integer-only reference kernels, GPU and CPU; test vectors
verifier/   committee and watcher client: sample, fetch, recompute, commit,
reveal, challenge
sim/        sim.py and its extensions
spec/       this design, transaction encodings, dispute protocol
```

Testnet metrics

- **Useful share:** settled job compute divided by compute implied by difficulty, published every block.
- **λ per job type,** measured from miners' reported ticket rates against settled work.
- **Verification overhead:** verifier compute divided by job compute; the target is under 1%.
- **False-slash rate:** the target is zero.
- **Client price:** fee per unit of work compared with renting the same GPU time directly.

Sources

The fourteen papers are linked in the table under Where each paper lands. Their content comes from the full-text reads in the literature review. Opened for this version:

- Pearl repository: zk proof-of-work circuit, XMSS, vLLM miner

- Basu, The Usefulness Gap: the {−64, …, 64} entry range, the verifier not checking matrix origin, the dominant miner with no identifiable inference code, the value destruction ratio
- Teutsch and Reitwießner, Truebit: forced errors and jackpots, about one forced error per thousand tasks
- Verde: refereed delegation and reproducible operators (RepOps)
- I-LLM: integer-only LLM inference at W4A4

Earlier drafts of this design are in Useful-Work Coin: Implementation Design and the literature review.